

Fine-tuning Bodhan AI Indic-Transcribe for Marathi ASR with Low Rank Adaptation (LoRA)

Take-home assignment, AI Research Engineer, AI4Bharat (IIT Madras) Author: **Ankita** Artifacts (checkpoint, logs, per-utterance results):

Google Drive link: <https://drive.google.com/drive/folders/1-5SH7LvjG26wtqQxBQ5bQGJ89LGvtokJ?usp=sharing>

1. Summary

I fine-tuned **Indic-Transcribe-Flex** (Bodhan AI / AI4Bharat, 1.22B parameters, NVIDIA Canary architecture) on **Marathi** using **LoRA**, on a single free Colab T4 GPU.

The whole pretrained model stays frozen. Low-rank adapters (41.9M trainable parameters, 3.4% of the model) are injected into the encoder's self-attention and feed-forward layers and the decoder's self- and cross-attention. Training data is a 1,000 utterances subset of FLEURS Marathi (~3.3 h of read speech). On the held-out FLEURS Marathi test set, WER improved from 18.24% to 15.23% (16% relative) and CER from 6.42% to 4.51% (30% relative), after 46.7 minutes of training with a 10.2 GB GPU memory peak.

As a further analysis beyond the task, I applied the resulting adapter to a language it was never trained on, to measure how far the adaptation generalises. Details are given in section 6.3.

2. Model and data-set

Model

The encoder is a 32-layer FastConformer with approximately 840 million parameters. Each layer consists of two macaron style feed-forward blocks, a multi-head self-attention block with relative positional encoding and a convolution module containing batch normalisation. The model dimension is 1,024, the feed-forward dimension is 4,096 and the self-attention uses eight heads. The decoder is a 24-layer Transformer with approximately 423 million parameters. Each layer contains a self-attention sub-layer over the tokens generated so far, a cross-attention sub-layer that attends to the encoder output and a feed-forward sub-layer.

Input features are 128-dimensional log-mel filterbank coefficients computed inside the model from 16 KHz monaural audio and the output vocabulary is an aggregate SentencePiece model of 7,152 sub-word units. The task the model performs is determined by prompt tokens rather than by separate output heads. Every example carries a source language, a target language and a flag requesting punctuation and capitalisation. Setting the source and target languages to the same value specifies transcription, whereas setting them to different values would specify translation.

Two checkpoints are published. Core is intended for native-script output, while Flex additionally supports romanised and mixed-script transcripts. I used Flex because its documentation includes the NeMo checkpoint and the loader on which training depends.

Data-set

The training data is the Marathi portion of FLEURS, obtained from the Hugging Face Hub under the configuration name `mr_in`. FLEURS is a corpus of read speech with transcripts produced by human annotators, released under a Creative Commons Attribution 4.0 licence. It provides official training, validation and test partitions that are separated by sentence, so that no sentence occurring in the test partition appears anywhere in training. The corpus was recently republished in the Parquet format, which allows it to be loaded by current versions of the data-sets library without a custom loading script.

From these partitions I drew a random subset, fixed by a random seed so that the same selection is obtained on every run. Training uses 1000 utterances, amounting to approximately 3.3 hours of speech; validation uses 150 utterances and testing uses 300, of which 297 remain after filtering. The subset is drawn at random rather than taken as the first rows of the corpus, because the rows are ordered by sentence and by speaker, so that a prefix of the corpus would over-represent a small number of sentences and voices. The size of the sub-set governs the volume downloaded, the duration of an epoch and the time required for evaluation, but it does not govern the memory consumed on the graphics processor, which is determined instead by the total audio duration packed into each batch.

Each recording is decoded with the soundfile library. The audio is converted to a single channel at 16 KHz and written as 16-bit linear PCM. Utterances shorter than half a second are discarded, since they are usually noise or truncations and utterances longer than thirty seconds are discarded because thirty seconds is the longest input window the model accepts; three test utterances were removed for this reason. The transcripts are normalised to Unicode Normalization Form C, their whitespace is collapsed and their punctuation is retained.

Punctuation is retained because the prompt supplied at inference requests punctuation and capitalisation. FLEURS also provides a normalised transcript column from which punctuation has been stripped, but training on that column while continuing to request punctuation would teach the model to contradict its own prompt. Punctuation is instead removed from both the reference and the hypothesis at the moment of scoring, so that retaining it in training does not inflate the reported error rate.

A further 100 Hindi utterances, drawn from the test partition of the hi_in configuration, were prepared for the analysis reported in section 6.3. They are used only for evaluation and never for training.

FLEURS is a widely used public data-set and may overlap with the material on which the base model was trained. This is the reason the base model is measured on the same test set rather than relying on the figure published in the model card, and it is the reason an evaluation on out-of-domain Marathi speech is named as the next experiment to run.

3. Approach and the reasoning behind each decision

Training through NeMo rather than the Hugging Face wrapper

The repository ships two entry points. The Hugging Face one (`IndicTranscribe.from_pretrained`) is **inference-only**: it maps audio to text and exposes no loss, no label handling and no teacher forcing. The same weights also ship as an NVIDIA NeMo checkpoint (`nemo/` folder) with a loader that registers the model's custom tokenizer.

NeMo is the framework Bodhan used to train the model, so it already provides the loss, prompt construction, Lhotse data loading and validation metrics; all matching how the base was trained. Reimplementing the prompt format and loss around an inference wrapper would be slower to build and easy to get subtly wrong in ways that train silently but incorrectly.

LoRA as the fine-tuning method

With ~3 hours of in-language data against a model trained on ~1.3M hours, the goal is a small, constrained adjustment, not retraining. LoRA learns a low-rank update, $W' = W + (\alpha/r) \cdot B \cdot A$, with B initialised to zero so training starts exactly at the pretrained model, and leaves every pretrained weight untouched.

Three concrete reasons here:

1. **Overfitting and forgetting:** A rank-constrained update is a strong regulariser on a small data-set and the base weights are recoverable by construction.
2. **Memory:** Full fine-tuning with AdamW needs ~16 bytes per trainable parameter (~19 GB before activations), which does not fit a 15 GB T4. With LoRA the optimiser state is negligible and memory goes to activations; the run peaked at 10.2 GB.
3. **Deployment.** The adapter is small enough to checkpoint repeatedly and can be merged into the weights for zero-overhead inference or kept per language on a shared frozen base.

Configuration: Rank 32, alpha 64 (scaling 2), dropout 0.05; the settings I validated in my Whisper children's ASR experiments. Targets: encoder self-attention (q, k, v, out), both encoder feed-forward blocks, decoder self-attention, decoder cross-attention. Encoder adapters handle acoustic adaptation; decoder self-attention handles Marathi word forms and orthographic conventions; cross-attention handles audio-text alignment. The convolution modules (Conv1d) and the relative-position projection are deliberately not adapted.

LoRA implemented directly, not via peft

peft is built around Hugging Face PreTrainedModels: `get_peft_model`, `save_pretrained` and `merge_and_unload` all assume that interface. This is a NeMo LightningModule, so I replaced the target `nn.Linear` layers in place with a `LoRALinear` wrapper. The model stays a standard `EncDecMultiTaskModel`, so NeMo's training step, validation, `transcribe()` and export keep working

unchanged. Target selection is by regex over module names, training refuses to start if nothing matches and a warning is printed for any target group that matches zero layers eventually, a silent "zero trainable parameters" run is not possible.

Frozen network kept in eval mode

Setting requires_grad=False stops gradient updates but does **not** stop BatchNorm running statistics from updating in training mode and every FastConformer convolution module contains a BatchNorm. Lightning puts the model back into training mode at each epoch and after every validation, so the "frozen" network would silently drift towards a few hours of FLEURS statistics.

A callback therefore puts every module holding pretrained weights into eval mode before each training batch and re-enables only the LoRA dropout. The top-level model deliberately stays in training mode, because NeMo applies SpecAugment only then; SpecAugment plus LoRA dropout is the regularisation used on this small data-set.

This did not arise in my earlier Whisper LoRA work, since Whisper uses LayerNorm only. It is architecture-specific and easy to miss.

Optimisation

AdamW, lr 2e-4, betas (0.9, 0.98), **no weight decay** (decaying a low-rank update towards zero is decaying towards "no adaptation"), 10% linear warmup then cosine decay to 1e-6, gradient clipping 1.0, 2 epochs = 228 optimiser steps. fp16 mixed precision with fp32 master weights: the checkpoint is published in fp16, and updates at lr 2e-4 are lost to rounding if applied directly to fp16 parameters.

Dynamic batching: clips range from under 1 s to 30 s, so batches are filled to 40 s of audio (3 accumulation steps $\approx$ 120 s per update) rather than to a fixed clip count, with bucketing so less compute is spent on padding. The step budget is derived from total training audio, so the same configuration behaves sensibly on any GPU.

Checkpointing

A full Lightning checkpoint of this model is several GB. Instead, only the LoRA tensors are saved (tens of MB) whenever **validation WER** improves, together with the LoRA configuration.

4. Results

I have used word error rate (WER) and character error rate (CER) as evaluation metric. Reference and hypothesis are normalised identically before scoring (Normalization Form C (NFC), zero-width characters were removed, all unicode punctuation were also removed whereas whitespace collapsed). Both models are decoded with the same call and prompt: source_lang="mr", target_lang="mr", pnc="yes" (source_lang stands for source language, target_lang stands for target language, mr for Marathi and pnc for punctuation).

Results for Marathi test set (297 utterances)

Model	Trainable params	WER	CER
Indic-Transcribe-Flex, base model	0	18.24%	6.42%
+ LoRA adapter (this work)	41.9M (3.4%)	15.23%	4.51%
Absolute improvement		3.01	1.91
Relative improvement		16.5%	29.7%

Further analysis: Applying the Marathi adapter to a non-target language (section 6.3)

Condition	Hindi WER	Hindi CER
Base model (matched)	11.85%	6.20%
+ Marathi adapter (mis-matched)	14.08%	8.40%
Relative improvement	18.8%	35.5%

Hindi: I have used 100 test utterances which were never used for training purpose.

Run details: Tesla T4 (15 GB), fp16 mixed precision, 228 optimiser steps (2 epochs), best checkpoint at step 57 (validation WER 0.2083), training time 46.7 min, peak GPU memory 10.2 GB. It is worth mentioning here that the base model row is often called "zero-shot", but Indic-Transcribe was trained on Marathi already. It is more precisely "the base model with no task-specific fine-tuning".

(Per-utterance results for every evaluation are saved as CSV in the artifacts folder.)

5. problems faced

- val_loss was NaN under fp16** while val_wer computed normally. Instrumenting the training loop showed 0 NaN/inf losses in 680 training batches, so training itself was healthy and only the validation-loss reduction overflowed. Fix: select checkpoints by validation WER, which is the of interest anyway.
- The session crashed at the end of every run, reproducibly.** PyTorch Lightning's default teardown copies the entire model (~5 GB) from GPU to system RAM when fit() ends; free Colab has ~12.7 GB of RAM and NeMo plus the data loaders already occupy much of it. Fix: a SingleDeviceStrategy subclass that skips that copy (the model is needed on the GPU for evaluation anyway).
- Untokenizable characters in the references.** Some FLEURS text contains the zero-width joiner and digits the tokenizer cannot represent; they round-trip to <unk>, and training on <unk> teaches the model to emit it. Fix: strip the invisible joiners, and exclude the remaining affected sentences from **training only**; the test set is untouched, so the comparison stays fair.
- BatchNorm drift in the frozen encoder**, found by reasoning about the architecture rather than from a failure; it would have degraded the frozen model silently.
- FLEURS id is not unique per recording:** It is a sentence id shared by every speaker who read that sentence. Fix: generate utterance ids from the row index.

6. Reproducing

- All work runs in a single Colab notebook (notebooks/), top to bottom: setup → data preparation → base-model evaluation → LoRA injection → smoke test → training → evaluation → comparison. It needs a Hugging Face token with the model licence accepted, stored in Colab Secrets and a mounted Google Drive for artifacts.
- To rebuild the fine-tuned model: download the public base checkpoint, inject LoRA with the configuration stored inside `best_lora.pt` and load that file's weights.
- Artifacts** (Google Drive): training log, TensorBoard and CSV logs, validation history, `best_lora.pt`, `summary.json`, and per-utterance CSV/JSON results for all four evaluations.